Q:- 1. Database Initialization and Table Creation: Create a New Database, and create tables: students table, courses table & enrollments table.

- **Input:-**
- **init_tables.py:-**

```
import sqlite3

def init_database():
    # 1. Connect to SQLite database (creates database file if it does not exist)
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # Enable Foreign Key Support
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 2. Create 'students' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        student_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER NOT NULL,
        major TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL
    );
    ''')

    # 3. Create 'courses' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS courses (
        course_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_code TEXT UNIQUE NOT NULL,
        course_name TEXT NOT NULL,
        credits INTEGER NOT NULL
    );
    ''')

    # 4. Create 'enrollments' table with Foreign Keys
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS enrollments (
        enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id INTEGER NOT NULL,
        course_id INTEGER NOT NULL,
        grade TEXT,
```

```
        FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
RESTRICT,
        FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE
    );
    "")

    conn.commit()
    print("Database 'university.db' initialized successfully.")
    print("Tables created: students, courses, enrollments.")
    conn.close()

if __name__ == '__main__':
    init_database()
```

- **Output:-**

```
Database 'university.db' initialized successfully.
Tables created: students, courses, enrollments.
```

Q:- 2. Data Insertion (Create Operations): Insert Sample Data into students, courses, and enrollments tables.

- **Input:-**
- **insert_sample_data.py:-**

```
import sqlite3

def insert_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Insert sample students
    students_data = [
        ('Amit Sharma', 20, 'Computer Science', 'amit.sharma@example.com'),
        ('Priya Patel', 21, 'Information Technology', 'priya.patel@example.com'),
        ('Rahul Verma', 22, 'Electronics', 'rahul.verma@example.com'),
        ('Sneha Joshi', 20, 'Computer Science', 'sneha.joshi@example.com')
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO students (name, age, major, email)
    VALUES (?, ?, ?, ?);
    ''', students_data)

    # Insert sample courses
    courses_data = [
        ('CS501', 'Database Systems', 4),
        ('CS504', 'Web Development Using MEAN Stack', 4),
        ('IT502', 'Cloud Computing', 3)
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO courses (course_code, course_name, credits)
    VALUES (?, ?, ?);
    ''', courses_data)

    # Insert sample enrollments
    enrollments_data = [
        (1, 1, 'A'),
        (1, 2, 'B'),
        (2, 2, 'A'),
        (3, 3, 'B'),
        (4, 1, 'A')
    ]
```

```
    cursor.executemany('''
    INSERT INTO enrollments (student_id, course_id, grade)
    VALUES (?, ?, ?);
    ''', enrollments_data)

    conn.commit()
    print("Sample data inserted successfully into students, courses, and enrollments.")
    conn.close()

if __name__ == '__main__':
    insert_data()
```

- **Output:-**

Sample data inserted successfully into students, courses, and enrollments.

Q:- 3. Data Retrieval (Read Operations): 1. Basic Retrieval, 2. Filtered Retrieval, 3. Complex Queries (using JOINs).

- **Input:-**
- **retrieve_data.py:-**

```
import sqlite3

def retrieve_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Basic Retrieval - All Students
    print("--- 1. Basic Retrieval (All Students) ---")
    cursor.execute("SELECT student_id, name, major, email FROM students;")
    for row in cursor.fetchall():
        print(f"ID: {row[0]} | Name: {row[1]} | Major: {row[2]} | Email: {row[3]}")

    # 2. Filtered Retrieval - Students with Computer Science Major
    print("\n--- 2. Filtered Retrieval (Major = 'Computer Science') ---")
    cursor.execute("SELECT name, email FROM students WHERE major = 'Computer
Science';")
    for row in cursor.fetchall():
        print(f"Student: {row[0]} | Email: {row[1]}")

    # 3. Complex Query - JOIN students, enrollments, and courses
    print("\n--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---")
    query = '''
    SELECT s.name, c.course_code, c.course_name, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id
    ORDER BY s.name;
    '''
    cursor.execute(query)
    for row in cursor.fetchall():
        print(f"Student: {row[0]:<14} | Course: {row[1]} ({row[2]}) | Grade: {row[3]}")

    conn.close()

if __name__ == '__main__':
    retrieve_data()
```

➢ Output:-

```
--- 1. Basic Retrieval (All Students) ---
ID: 1 | Name: Amit Sharma | Major: Computer Science | Email: amit.sharma@example.com
ID: 2 | Name: Priya Patel | Major: Information Technology | Email: priya.patel@example.com
ID: 3 | Name: Rahul Verma | Major: Electronics | Email: rahul.verma@example.com
ID: 4 | Name: Sneha Joshi | Major: Computer Science | Email: sneha.joshi@example.com

--- 2. Filtered Retrieval (Major = 'Computer Science') ---
Student: Amit Sharma | Email: amit.sharma@example.com
Student: Sneha Joshi | Email: sneha.joshi@example.com

--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---
Student: Amit Sharma   | Course: CS501 (Database Systems) | Grade: A
Student: Amit Sharma   | Course: CS504 (Web Development Using MEAN Stack) | Grade: B
Student: Priya Patel   | Course: CS504 (Web Development Using MEAN Stack) | Grade: A
Student: Rahul Verma   | Course: IT502 (Cloud Computing) | Grade: B
Student: Sneha Joshi   | Course: CS501 (Database Systems) | Grade: A
```

Q:- 4. Data Update (Update Operations): 1. Update Student Major, 2. Update Enrollment Grade.

- **Input:-**
- **update_data.py:-**

```
import sqlite3

def update_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Update Student Major
    cursor.execute('''
    UPDATE students
    SET major = 'Data Science'
    WHERE name = 'Amit Sharma';
    ''')
    print("1. Student Major Updated: Amit Sharma changed to Data Science.")

    # 2. Update Enrollment Grade
    cursor.execute('''
    UPDATE enrollments
    SET grade = 'A+'
    WHERE student_id = 1 AND course_id = 2;
    ''')
    print("2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.")

    conn.commit()

    # Verification Query
    print("\nVerification of Updated Records:")
    cursor.execute("SELECT name, major FROM students WHERE name = 'Amit Sharma';")
    print("Updated Student:", cursor.fetchone())

    cursor.execute("SELECT student_id, course_id, grade FROM enrollments WHERE
student_id = 1 AND course_id = 2;")
    print("Updated Enrollment:", cursor.fetchone())

    conn.close()

if __name__ == '__main__':
    update_operations()
```

➢ Output:-

1. Student Major Updated: Amit Sharma changed to Data Science.
2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.

Verification of Updated Records:
Updated Student: ('Amit Sharma', 'Data Science')
Updated Enrollment: (1, 2)

Q:- 5. Data Deletion (Delete Operations): 1. Delete an Enrollment, 2. Attempt to Delete Enrolled Student (Observe FK Constraint), 3. Safe Student Deletion.

- **Input:-**
- **delete_operations.py:-**

```
import sqlite3

def delete_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 1. Delete an Enrollment
    cursor.execute("DELETE FROM enrollments WHERE enrollment_id = 1;")
    conn.commit()
    print("1. Enrollment with ID 1 deleted successfully.")

    # 2. Attempt to Delete Enrolled Student (Observe FK Constraint)
    print("\n2. Attempting to delete student ID 2 who has active enrollments...")
    try:
        cursor.execute("DELETE FROM students WHERE student_id = 2;")
        conn.commit()
    except sqlite3.IntegrityError as e:
        print(f"FOREIGN KEY CONSTRAINT CAUGHT: {e}")
        print("Observation: The database rejected deletion because RESTRICT constraint
protects enrolled students.")

    # 3. Safe Student Deletion: Delete student's enrollments first, then student
    print("\n3. Executing Safe Student Deletion for Student ID 3:")
    cursor.execute("DELETE FROM enrollments WHERE student_id = 3;")
    cursor.execute("DELETE FROM students WHERE student_id = 3;")
    conn.commit()
    print("Enrollments for student 3 removed, followed by safe student deletion.")

    conn.close()

if __name__ == '__main__':
    delete_operations()
```

- **Output:-**

```
1. Enrollment with ID 1 deleted successfully.
```

2. Attempting to delete student ID 2 who has active enrollments...
FOREIGN KEY CONSTRAINT CAUGHT: FOREIGN KEY constraint failed
Observation: The database rejected deletion because RESTRICT constraint protects enrolled students.

3. Executing Safe Student Deletion for Student ID 3:
Enrollments for student 3 removed, followed by safe student deletion.

Q:- 6. Database Dumping: 1. Full Database Dump, 2. Schema-Only Dump, 3. Data-Only Dump.

- **Input:-**
- **database_dumps.py:-**

```
import sqlite3

def dump_database():
    conn = sqlite3.connect('university.db')

    # 1. Full Database Dump (Schema + Data)
    with open('full_dump.sql', 'w') as f:
        for line in conn.iterdump():
            f.write(f"{line}\n")
    print("1. Full Database Dump exported to 'full_dump.sql'.")

    # 2. Schema-Only Dump
    cursor = conn.cursor()
    cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name NOT
LIKE 'sqlite_%';")
    schema_statements = cursor.fetchall()
    with open('schema_dump.sql', 'w') as f:
        for stmt in schema_statements:
            if stmt[0]:
                f.write(f"{stmt[0]};\n\n")
    print("2. Schema-Only Dump exported to 'schema_dump.sql'.")

    # 3. Data-Only Dump (INSERT statements)
    with open('data_dump.sql', 'w') as f:
        for line in conn.iterdump():
            if line.startswith('INSERT INTO'):
                f.write(f"{line}\n")
    print("3. Data-Only Dump exported to 'data_dump.sql'.")

    conn.close()

if __name__ == '__main__':
    dump_database()
```

- **Output:-**

```
1. Full Database Dump exported to 'full_dump.sql'.
2. Schema-Only Dump exported to 'schema_dump.sql'.
3. Data-Only Dump exported to 'data_dump.sql'.
```

Q:- 7. CSV File Handling: 1. Export Enrollments to CSV, 2. Import New Students from CSV.

- **Input:-**
- **csv_handling.py:-**

```
import sqlite3
import csv

def handle_csv():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Export Enrollments to CSV
    cursor.execute('''
    SELECT e.enrollment_id, s.name, c.course_code, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id;
    ''')
    rows = cursor.fetchall()
    with open('enrollments_export.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['Enrollment_ID', 'Student_Name', 'Course_Code', 'Grade'])
        writer.writerows(rows)
    print("1. Enrollments exported successfully to 'enrollments_export.csv'.")

    # 2. Import New Students from CSV
    # Create sample import CSV
    new_students = [
        ['Vikram Malhotra', 21, 'Cyber Security', 'vikram.m@example.com'],
        ['Ananya Sen', 20, 'Data Science', 'ananya.s@example.com']
    ]
    with open('new_students.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['name', 'age', 'major', 'email'])
        writer.writerows(new_students)

    with open('new_students.csv', 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            cursor.execute('''
            INSERT OR IGNORE INTO students (name, age, major, email)
            VALUES (?, ?, ?, ?);
```

```
            ''', (row['name'], row['age'], row['major'], row['email']))

    conn.commit()
    print("2. New students imported successfully from 'new_students.csv'.")

    # Verification
    cursor.execute("SELECT student_id, name, major FROM students WHERE major IN
('Cyber Security', 'Data Science');")
    print("\nNewly Imported Students Verified:")
    for r in cursor.fetchall():
        print(f"ID: {r[0]} | Name: {r[1]} | Major: {r[2]}")

    conn.close()

if __name__ == '__main__':
    handle_csv()
```

- **Output:-**

```
1. Enrollments exported successfully to 'enrollments_export.csv'.
2. New students imported successfully from 'new_students.csv'.

Newly Imported Students Verified:
ID: 1 | Name: Amit Sharma | Major: Data Science
ID: 5 | Name: Vikram Malhotra | Major: Cyber Security
ID: 6 | Name: Ananya Sen | Major: Data Science
```